ProtomegaTron as a Hyperseed-Scrutable Agent Loop

OmegaClaw / ZeroBot working notes for Ben Goertzel

2026-06-27

Abstract

This note begins a declarative Hyperseed-style formalization of ProtomegaTron: a private OmegaClaw agent persona running through the OmegaClaw MeTTa/Python loop, using OpenClaw as an LLM backend, Telegram as an interaction channel, ChromaDB as long-term associative memory, and PeTTa/MeTTa structures for symbolic control and reasoning. The aim is not to anthropomorphize the system, but to make its design and dynamics scrutable as a structured, provenance-bearing process.

1 Provenance and code anchors

This note is grounded in the local OmegaClaw checkout inspected on 2026-06-27:

- OmegaClaw Core commit `16d380d`, with local ProtomegaTron/OpenClaw patches.
- Loop and scheduling: `src/loop.metta`.
- Memory and prompt selection: `src/memory.metta`.
- Channel dispatch and anti-duplicate send: `src/channels.metta`.
- Telegram adapter and access controls: `channels/telegram.py`.
- Skill grammar and MeTTa/IO/communication actions: `src/skills.metta`.
- OpenClaw provider bridge: `lib_llm_ext.py`.
- ProtomegaTron mandate: `memory/prompt_OpenClaw.txt`.

2 A situated-process account

Definition 1 (ProtomegaTron process state)**.** *A ProtomegaTron process state is a tuple*

$$P_t = \langle M_t, C_t, L_t, S_t, H_t, B_t, \Pi_t, A_t, R_t, V_t \rangle$$

where M_t is the mandate/persona prompt, C_t the active channel context, L_t the loop-control state, S_t the skill/action grammar, H_t memory and history, B_t backend model/provider state, Π_t policies and access controls, A_t emitted actions, R_t observations/results, and V_t provenance.

Definition 2 (Hyperseed-scrutable agent loop)**.** *A software agent loop is* Hyperseed-scrutable *to the degree that its state components, transition rules, action affordances, memory updates, failures, and provenance can be represented as inspectable atoms or typed records, rather than remaining only implicit in code and transient prompt text.*

3 The OmegaClaw turn transition

The inspected loop implements the following high-level transition:

$$P_{t+1} = T(P_t, I_t)$$

with stages:

$$\begin{aligned}
I_t &\leftarrow \text{receive}(C_t) \\
F_t &\leftarrow \text{fresh?}(I_t, \text{prevmsg}_t) \\
X_t &\leftarrow \text{context}(M_t, S_t, H_t, R_{t-1}, \text{time}_t) \\
Y_t &\leftarrow \text{LLM}(B_t, X_t, I_t) \\
Z_t &\leftarrow \text{parse_sexpr}(Y_t) \\
R_t &\leftarrow \text{execute}(Z_t, S_t) \\
H_{t+1} &\leftarrow \text{append_history}(H_t, I_t, Y_t, Z_t, R_t) \\
L_{t+1} &\leftarrow \text{update_loop}(L_t, F_t, R_t).
\end{aligned}$$

Example 1 (Fresh-message path)**.** *When Telegram receives a new allowed private message from Ben, the adapter exposes a nonempty message. The loop detects novelty relative to* `&prevmsg`*, resets* `&loops` *to* `maxNewInputLoops`*, constructs a prompt containing mandate, skills, last skill results, history tail, and time, calls the OpenClaw provider, parses returned skill calls such as* `(send ...)`*, executes them, and appends an episodic record to* `memory/history.metta`*.*

Example 2 (Idle/no-input path)**.** *With the local quiet-start patch, initialization sets* `&loops=0`*. If no fresh message arrives, the loop sleeps and recurs without an LLM call unless an explicitly configured autonomous wake loop is due. This converts "waiting for input" from an unbounded backend-spending process into a gated attentional process.*

4 Agency boundaries as constraints

Definition 3 (Action boundary)**.** *An action boundary is a predicate $\beta(P_t, a)$ determining whether action a is admissible in process state P_t. In the current system, relevant boundaries include Telegram allowlists/private-only checks, authentication binding, security policy, provider availability, duplicate-send suppression, prompt instruction to use* `send` *only for meaningful updates, and loop gating through* `&loops`*.*

Proposition 1 (Boundary visibility improves scrutability)**.** *If action boundaries are represented declaratively and logged with provenance, then unexpected behavior can be localized to either input classification, boundary evaluation, LLM action proposal, parser repair, skill execution, or memory update.*

Proof sketch. The loop already separates these stages operationally. Making each stage emit typed records preserves the causal path from inbound message to outbound action. A failure or surprise can then be inspected by querying records for the corresponding stage rather than reconstructing intent from raw logs alone. □

5 Memory stratification and the proposed intermediate PeTTa store

The inspected memory design currently has several levels:

1. Working memory / `pin`: immediate task state.
2. Episodic trace: `memory/history.metta`, tail-injected into prompts.
3. ChromaDB long-term memory: semantic associative recall through `remember`/`query`.
4. Per-invocation AtomSpace reasoning: powerful but not persistent across calls.

Definition 4 (Medium-scale PeTTa memory)**.** *A medium-scale PeTTa memory is a persistent or session-persistent AtomSpace-like store*

$$K_t^{mid} = \{\alpha_i\}_{i=1}^{n}$$

where each α_i is a typed atom or small atom cluster about recent episodes, entities, commitments, hypotheses, open questions, action boundaries, and provenance. It is more structured than embedding memory and less permanent than curated long-term records.

Candidate atoms include:

```
(Episode e123)
(Said Ben e123 "progress with semantic formalization")
(About e123 ProtomegaTronDesign)
(OpenQuestion q77 "How should medium PeTTa memory decay?")
(HasStatus q77 active)
(ConstrainedBy send-action meaningful-update-policy)
(Confidence (Useful MediumPeTTaMemory) (stv 0.8 0.6))
```

Design Principle 1 (Selective atomization)**.** *Do not atomize every token of experience. Atomize high-value events: explicit user commitments, task state, design hypotheses, observed failures, decisions, provenance, and concepts likely to participate in later reasoning.*

Conjecture 1 (Intermediate-memory benefit)**.** *Adding K^{mid} between working memory and ChromaDB will improve continuity and self-scrutability for ProtomegaTron if it remains selective, typed, provenance-bearing, and periodically distilled into long-term memory or project records.*

Conjecture 2 (Schema-sprawl risk)**.** *A medium PeTTa memory that grows without a small ontology of episode, commitment, hypothesis, question, boundary, and provenance atoms will reduce scrutability by creating unreliable symbolic clutter.*

6 Three selves

Definition 5 (Code-level self)**.** *The code-level self is the MeTTa/Python transition system: loop, memory, channel, provider, skills, security, and process supervision.*

Definition 6 (Prompt-level self)**.** *The prompt-level self is the mandate/persona/operational instruction surface, especially `prompt_OpenClaw.txt`: ProtomegaTron as Ben's concise, Hyperseed-oriented, non-spamming OmegaClaw research assistant.*

Definition 7 (Historical self)**.** *The historical self is the accumulated trace of episodes, remembered facts, task records, commits, and formalizations that constrain and inform future behavior.*

Proposition 2 (Non-homuncular self model)**.** *ProtomegaTron need not be modeled as a hidden inner entity. It can be modeled as a recursively scrutable process whose apparent identity is the relatively stable pattern across code-level transition rules, prompt-level mandates, historical records, and current interaction context.*

7 Next formalization steps

1. Extract a minimal atom schema for episodes, commitments, hypotheses, questions, boundaries, and provenance.
2. Encode one Telegram fresh-message episode and one idle/no-input episode using that schema.
3. Specify a prototype `medium-memory.metta` API: append, query-by-type, query-by-entity, decay/distill, and export-to-Chroma/project-notes.
4. Compare this declarative account against actual OmegaClaw logs from a supervised smoke run.

8 Open questions for Ben

1. Should K^{mid} be primarily an engineering feature for ProtomegaTron, or also a philosophical exemplar of Hyperseed self-formalization?
2. Should the first prototype live directly inside OmegaClaw Core, or as a local experimental patch in the research workspace until the schema stabilizes?
3. How much of the medium-memory schema should reuse existing Hyperseed primitives versus pragmatic OmegaClaw-specific predicates?